

SDAIA · Modern Data Engineering for AI Systems

Healthcare Clinical Trial Matcher

An integrated AI data platform that answers clinical questions from real registry evidence — grounded, private, and fully auditable.

DAY 5 CAPSTONE

Architecture Integration

IMPLEMENTATION

Single file · 2,081 lines

Dana Wael Alhelal

Course reference SDAIA-F-CRS-100-01-V1 · github.com/Danaaaao/Eng

A chatbot cannot do this job

Asking a general-purpose LLM about clinical trials fails in three specific ways

01

It invents evidence

Asked for trials matching a patient, an LLM returns confident NCT identifiers and eligibility thresholds that do not exist. In a clinical setting that is not a bad answer — it is a dangerous one.

02

It cannot see your data

Training data has a cutoff and never included the hospital's own patient records. The trials it 'knows' may have closed years ago.

03

It leaves no trace

No record of who asked what, which sources were used, or whether protected health information was exposed. Regulated environments cannot operate this way.

Three guarantees, enforced by code

A researcher asks in natural language. The platform answers only from real, validated, governed clinical trial records — and proves it.

Grounded

No invented trials or thresholds

- Retrieval over a curated, quality-gated corpus
- System prompt forbids outside knowledge
- Groundedness measured on every answer

Private

No PHI leaves the safe zone

- Identifiers stripped before embedding
- Vector database refuses any chunk carrying PHI
- Inbound questions redacted too

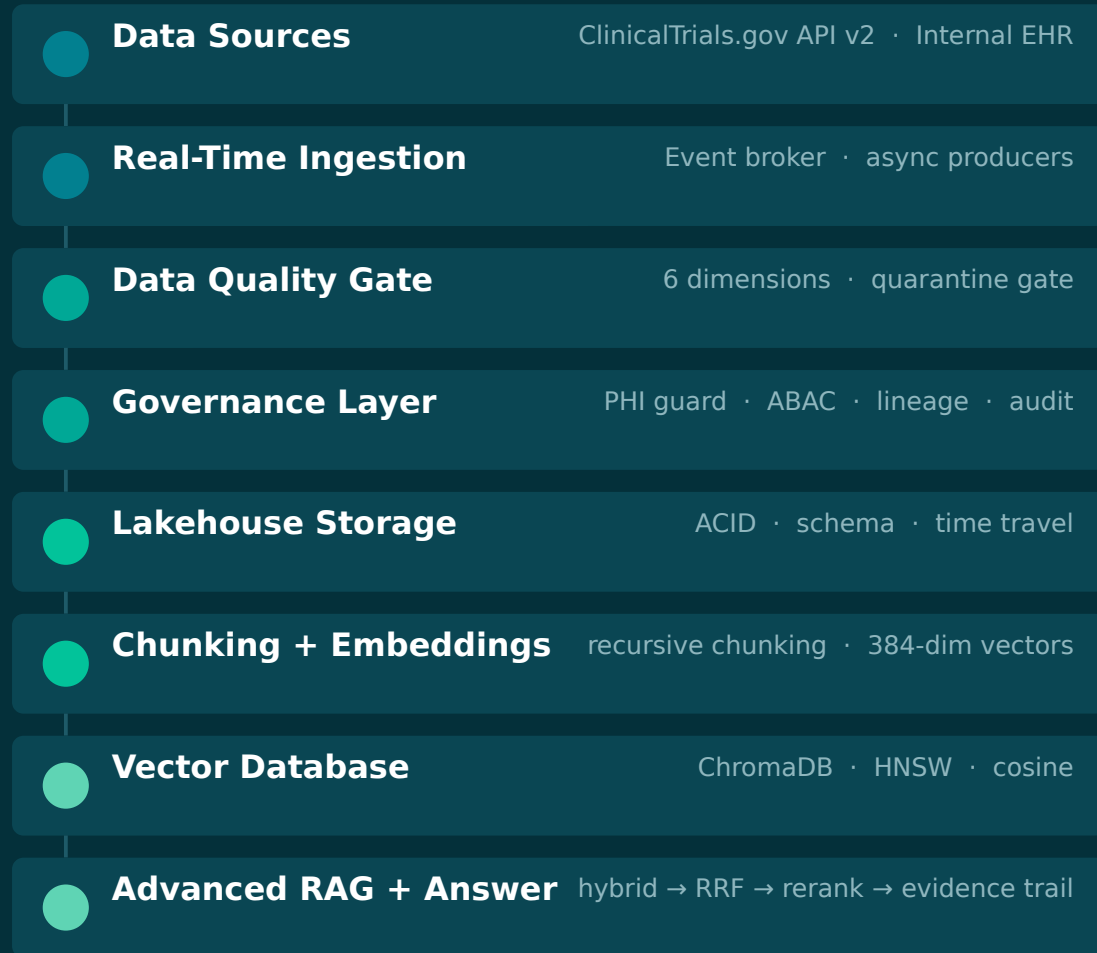
Auditable

Every access is on record

- Append-only audit trail per query
- End-to-end data lineage
- Evidence trail from claim to source

The integrated architecture

Eight layers, one platform — every component from the five course days



Orchestration DAG

All six pipeline stages run as a dependency-resolved graph with automatic retries, exponential backoff, and a per-component fallback for every external dependency.

The integration principle

Bad data is stopped before it becomes an embedding. Protected data is stripped before it becomes a vector. Every answer carries the provenance of the record it came from.

How the five days integrate

Each day of the course contributes a working layer, not a chapter of theory

1

Modern Data Architectures

`DeltaStyleLakehouse()`

Delta-style Lakehouse — ACID commits, schema enforcement, time travel, ELT raw preservation

2

Real-Time Data Pipelines

`MockEventBroker()`

Event-driven ingestion — async broker with topics, decoupled producers and consumers

3

Vector DBs & Advanced RAG

`HybridRetriever()`

Recursive chunking, embeddings, HNSW index, hybrid retrieval, cross-encoder reranking, RAG Triad

4

Quality & Governance

`PHIGuard()`

Six quality dimensions, HIPAA PHI guard, ABAC, lineage graph, append-only audit trail

5

Architecture Integration

`PipelineOrchestrator()`

Orchestration DAG with dependency resolution, retries, backoff, and fallbacks

Layer 1 — Ingestion

Day 2 · Event-driven pipeline over real registry data

- Real data: ClinicalTrials.gov public API v2 — no key, no scraping, authoritative source
- Two producers publish asynchronously to an event broker modelled on Kafka topic semantics
- Producers never wait for consumers — the decoupling the course calls loose coupling
- The untouched response is cached to the landing zone before any transformation (ELT)

Why a second source?

Registry data is public and carries no PHI. An internal EHR source — synthetic, but deliberately PHI-bearing — gives the governance layer something real to detect and block.

Three-tier resilience ladder

LIVE

Call the registry API, cache the raw response

REPLAY

Re-read the cached raw JSON — fully offline, identical results

SYNTHETIC

Clearly-labelled fallback records so a demo never dies

```
$ python clinical_trial_rag.py --stage pipeline
INFO  Fetched 4 studies (running total: 4)
INFO  Fetched 5 studies (running total: 9)
[PRODUCER] Ingestion mode = LIVE | 9 studies
```

Layer 2 — The quality gate

Day 4 · Six dimensions, two severities

Completeness

identifier & title present

FATAL

Validity

NCT identifier format

FATAL

Uniqueness

no duplicate trials

FATAL

Accuracy

ages and enrollment in range

FATAL

Consistency

status vocabulary

ADVISORY

Timeliness

updated within five years

ADVISORY

What testing changed

The first version quarantined every violation equally. Run against a realistic registry response, it rejected 5 of 9 records — mostly for being old — and halted the pipeline. But a completed trial from 2017 is still valid evidence.

FATAL

Record cannot be cited → quarantined, counts toward the 30% halt threshold

ADVISORY

Record kept, tagged, and the flag surfaces beside the evidence it qualifies

Layer 3 — HIPAA governance

Day 4 · Enforced by code, not by a policy document

A

De-identification at ingestion

Every RESTRICTED field — name, MRN, date of birth, phone, email, address — is dropped. Identity is replaced by a deterministic, irreversible pseudonym.

B

A hard gate at the vector database

Every chunk is re-scanned immediately before insertion. If an identifier survived, the write raises and the chunk never becomes a vector.

C

Query-time redaction and audit

A researcher pasting a patient name into the search box is the most common leak. Inbound questions are scrubbed and the stripped families are logged.

Proof, from the run

HIPAA GOVERNANCE - PHI DE-IDENTIFICATION

Patient records processed 3

PHI identifiers removed 18

Safe-zone fields: age, diagnosis,
hba1c, comorbidities, pseudonym

> attempting to index a poisoned chunk

X PHI detected (['EMAIL', 'MRN'])

Refusing to write it to the
vector database.

"No PHI in the vector DB" is a test that fails the build, not a sentence in a policy PDF.

Layer 4 — Lakehouse storage

Day 1 · ACID, schema enforcement, and time travel

ACID atomicity

The parquet file is written first and the commit entry appended only on success — a crash mid-write leaves an orphan no reader can ever see.

Schema enforcement

The blueprint is pinned at version 0. A batch with extra, missing, or retyped columns is rejected before it lands.

Time travel

Every commit is an immutable numbered entry in `_delta_log`, so any past version can be re-read exactly as it was.

Delta Lake semantics without a JVM — so the demo never fails on a Java version mismatch.

The transaction log, live

LAKEHOUSE TRANSACTION LOG

version	operation	numRecords	mode
0	WRITE	4	overwrite
1	APPEND	2	append

```
<< version 0 ..... 4 records
>> latest ..... 6 records
```

Schema drift, blocked

```
> appending an unannounced column
'investigator_notes' ...
X Transaction blocked by the Lakehouse!
Schema mismatch. Unexpected columns:
['investigator_notes']
```

Layers 5 & 6 — Building the corpus

Day 3 · Chunking, embeddings, and the vector index



Why HNSW and cosine

- Day 3's decision tree: high recall needed and RAM available → HNSW
- Cosine, so a long trial description cannot outrank a short precise one
- Chroma runs embedded — no server for an evaluator to start

A chunking decision worth explaining

Eligibility ages live in a metadata column, not in the criteria text — so our first build could never answer "what are the age limits?"

Fix: structured facts are inlined into eligibility chunks, making each chunk genuinely self-contained. That passage now ranks first for exactly that question.

Layer 7 — Advanced retrieval

Day 3 · Hybrid search, rank fusion, two-stage reranking

Dense search

HNSW · cosine · top 25

semantic meaning

Sparse search

BM25 · top 25

exact tokens

Reciprocal Rank Fusion · **RRF k = 60**

Merges two ranked lists without a shared score scale

Cross-encoder reranking · **top 5 survive**

The LLM reads five strong passages, not fifty

Why pure vector search is not enough

Embeddings are excellent on concepts and weak on exact tokens
— NCT04223752, HbA1c, SGLT2. BM25 covers precisely that blind spot.

```
stage-1 candidates : 21
  -> after RRF fusion
  -> after reranking
stage-2 survivors   : 5
retrieval latency   : 7.7 ms
```

Layer 8 — The answer, and its receipts

Every claim resolves to a governed source

Evidence trail

EVIDENCE TRAIL

```
[1] NCT05118932 | eligibility | via=hybrid
    chunk      : NCT05118932::eligibility::0
    lineage     : LIN_0003_LAKEHOUSE_WRITE
    source      : clinicaltrials.gov/study/...
[2] NCT02001010 | eligibility | via=hybrid
    ! quality: STALE_RECORD
    (last updated 2017-02-11)
```

Four identifiers per citation

- NCT ID — the trial in the public registry
- chunk ID — the exact passage retrieved
- lineage ID — the pipeline run that produced it
- quality flag — any advisory caveat on that record

Contradiction alerts

- ! Trial NCT05118932 declares a minimum age of 60 in its structured eligibility field, but its criteria text states 65 years. Verify against the registry before screening.

This is a real defect class in the registry. A researcher acting on the wrong value screens the wrong patients — so the platform surfaces the conflict instead of silently picking a side.

RAG Triad — measured on every answer

- Context Relevance — did retrieval pull the right passages?
- Groundedness — is the answer supported by them?
- Answer Relevance — does it address the question asked?

Orchestration — the conductor

Day 5 · Dependency resolution, retries, backoff, fallbacks



Dependency resolution

Topological sort — a task never starts before its inputs exist

Retries with backoff

Each task retries on failure, waiting 1s then 2s then 4s

Per-component fallbacks

Every external dependency has an offline equivalent

Run report

Status, attempts, and duration for every task — observability

Run report

task	status	attempts	seconds
ingest	SUCCESS	1	0.19
quality	SUCCESS	1	0.03
governance	SUCCESS	1	0.00
lakehouse	SUCCESS	1	0.03
chunk_embed	SUCCESS	1	5.88
vector_index	SUCCESS	1	0.69

If the platform is an orchestra, this is the conductor — the Day 5 analogy, implemented.

Running the platform

Five commands — each one maps to a layer you have just seen

--stage pipeline

Builds everything: ingests real registry data, runs the quality gate, applies governance, commits to the lakehouse, embeds, and indexes.

--stage query --q "..."

Asks a researcher question and returns the answer with its evidence trail, contradiction alerts, and RAG Triad scores.

--stage history

Prints the lakehouse transaction log and demonstrates time travel across versions.

--stage lineage

Renders the end-to-end data genealogy from source to vector index.

--stage demo

The full walkthrough — every layer, end to end. This is what we will run now.

Offline reproducibility: --stage pipeline --offline rebuilds the entire platform from the committed raw dataset, with no network access at all.

What we verified

Measured behaviour, not claimed behaviour

6 / 6

DAG tasks green

0.885

Groundedness score

18

PHI identifiers stripped

2,081

lines, one Python file

Verified end to end

- Two-page API pagination, raw caching, and identical offline replay
- Every seeded data defect landed in its intended quality dimension
- Schema drift blocked; time travel returned 4 then 6 records
- A PHI-bearing chunk was refused entry to the vector database
- A metadata-versus-text contradiction was detected and reported
- The generation call verified: model, temperature, prompt shape, parsing

Stated honestly

Our development environment blocked the live registry and model hosts at the network policy level. So the following remain unverified against the real services:

- The exact field names the live registry returns
- The real embedding and cross-encoder models

Both were exercised against fixtures built from the documented schema and the published model APIs. A mismatch would surface on the first live run.

Architectural decisions

Every substitution was a trade-off we can defend

Delta Lake → in-file Delta log

delta-spark needs Spark and a matching JVM. We reimplemented the semantics — ordered immutable commits, a pinned blueprint, write-then-commit atomicity — keeping the deliverable a single dependency-light file.

Qdrant → ChromaDB

Both use HNSW, so the retrieval architecture is unchanged. Chroma is in the Day 5 recommended stack, was taught on Day 3, and runs embedded with no server to start.

Naive RAG → hybrid + reranking

Dense-only retrieval failed on exact tokens and on questions whose answer lived in a metadata column. BM25 fusion and self-contained chunks fixed both.

A fully integrated platform moves an enterprise beyond simple AI prototypes — real-time streaming, strict governance, automated validation, and semantic retrieval working as a single machine.